МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

# Лабораторная работа №2
# по курсу «Операционные системы»

Выполнил: Г. П. Графов
Группа: М8О-203БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

## Условие

**Цель работы:** приобрести практические навыки управления процессами и организации межпроцессного обмена данными посредством каналов.

**Задание:** реализовать отдельные родительскую и дочернюю программы для Linux и Windows. Родитель должен создать дочерний процесс и организовать обмен данными через два канала. Ошибки системных вызовов должны обрабатываться.

**Вариант:** 5.

Родительский процесс первой строкой получает имя файла, а затем принимает по одному целому числу. Числа передаются дочернему процессу через первый канал. Дочерний процесс проверяет число на простоту. Составные числа записываются в файл. При получении отрицательного или простого числа дочерний процесс отправляет родителю сообщение через второй канал, после чего оба процесса завершаются.

## Метод решения

Работа выполнена на C++17. Стандартные средства языка используются для строк, разбора данных и управления временем жизни ресурсов, но процессы и каналы создаются прямыми системными вызовами. `std::thread`, `std::filesystem` и другие высокоуровневые обёртки над ОС не используются.

Родитель создаёт два однонаправленных канала. Первый передаёт строки с числами дочерней программе, второй возвращает односимвольный статус. После каждой команды родитель ожидает ответ, поэтому команды и статусы однозначно соответствуют друг другу.

В Linux вызов `fork()` создаёт новый процесс. В дочерней ветви `dup2()` переназначает читающий конец первого канала на стандартный ввод, а пишущий конец второго канала — на стандартный вывод. После этого `execl()` заменяет образ процесса программой `child`. Родитель использует `write()`, `read()` и `waitpid()`.

В Windows каналы создаются функцией `CreatePipe()`. Дескрипторы передаются дочернему процессу как `STDIN` и `STDOUT` через `STARTUPINFOA`; процесс создаётся функцией `CreateProcessA()`. Передача данных выполняется через `WriteFile()` и `ReadFile()`.

Проверка простоты перебирает нечётные делители до квадратного корня. Условие `divisor <= value / divisor` исключает переполнение. Значения 0 и 1 игнорируются, так как не являются ни простыми, ни составными.

## Описание программы

- `inc/parent.cpp` — создание каналов и процесса, пользовательский ввод, отправка команд, чтение статусов;
- `inc/child.cpp` — разбор чисел, проверка простоты, запись составных значений;
- `inc/common.cpp` — общие функции разбора и проверки простоты;
- `inc/include/protocol.hpp` — коды сообщений протокола;
- `CMakeLists.txt` — кроссплатформенная сборка двух программ.

Дескрипторы Linux и Windows помещаются в собственные RAII-классы `unique_fd` и `unique_handle`. Их деструкторы вызывают `close()` или `CloseHandle()`, поэтому ресурс освобождается при любом выходе из области видимости.

## Результаты

В результате выполнения лабораторной работы была разработана кроссплатформенная программа на языке C++17, состоящая из двух отдельных исполняемых файлов: родительского и дочернего процессов. Для обмена данными между процессами используются два однонаправленных канала.

Первый канал используется для передачи целых чисел от родительского процесса дочернему. Через второй канал дочерний процесс сообщает родительскому результат обработки очередного числа. Обмен выполняется последовательно: после отправки числа родитель ожидает ответ дочернего процесса.

Ключевые особенности полученного решения:

- родительский и дочерний процессы реализованы отдельными программами;
- процессы и каналы создаются непосредственными средствами операционной системы;
- для Linux используются системные вызовы `pipe()`, `fork()`, `dup2()`, `execl()`, `read()`, `write()` и `waitpid()`;
- для Windows используются функции `CreatePipe()`, `CreateProcessA()`, `ReadFile()`, `WriteFile()` и `WaitForSingleObject()`;
- дескрипторы автоматически освобождаются с помощью RAII-классов;
- некорректный ввод обрабатывается без аварийного завершения программы;
- средства `std::thread` и `std::filesystem` не используются.

Пример работы программы:

```
Output file: result.txt
Number: 4
Composite number was written to the file.
Number: 9
Composite number was written to the file.
Number: 1
0 and 1 are neither prime nor composite; value was ignored.
Number: 8
Composite number was written to the file.
Number: 7
Negative or prime number received. Processes are stopping.
```

После выполнения программы файл `result.txt` содержит:

```
4
9
8
```

Таким образом, составные числа записываются в указанный файл, значения 0 и 1 игнорируются, а при получении простого или отрицательного числа родительский и дочерний процессы корректно завершают работу.

## Выводы

В ходе работы было подтверждено, что обмен между независимыми процессами удобно представлять как передачу данных по отдельным направлениям. Разделение канала передачи чисел и канала ответных сообщений позволило исключить неоднозначность при обмене: родительский процесс отправляет очередное значение только в одном направлении, а результат его обработки получает в другом.

Также было установлено, что корректность взаимодействия процессов зависит не только от передачи данных, но и от управления временем жизни ресурсов. Закрытие неиспользуемых концов каналов необходимо для корректного обнаружения конца потока данных и предотвращения зависания процессов.

Реализованная программа показала, что одну и ту же схему межпроцессного взаимодействия можно сохранить для разных операционных систем, несмотря на различия их API. В Linux и Windows отличаются способы создания процесса и работы с дескрипторами, однако общая модель обмена данными через каналы остаётся одинаковой.

## Исходная программа

## Родительский процесс

```cpp
#include "protocol.hpp"

#include <array>
#include <cstdio>
#include <iostream>
#include <string>

#ifdef _WIN32
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#else
#include <limits.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#endif

namespace
{
    void print_status(char status)
    {
        if (status == protocol::written)
            std::cout << "Composite number was written to the file.\n";
        else if (status == protocol::ignored)
            std::cout << "0 and 1 are neither prime nor composite; value was ignored.\n
                ";
        else if (status == protocol::invalid)
            std::cout << "Child rejected the command; enter one integer.\n";
        else if (status == protocol::stop)
            std::cout << "Negative or prime number received. Processes are stopping.\n"
                ;
        else
            std::cerr << "Unknown child status.\n";
    }

#ifdef _WIN32
    class unique_handle
    {
    public:
        explicit unique_handle(HANDLE handle = nullptr) noexcept : handle_(handle) {}
        unique_handle(const unique_handle &) = delete;
        unique_handle &operator=(const unique_handle &) = delete;
        ~unique_handle() { reset(); }
        HANDLE get() const noexcept { return handle_; }
        void reset(HANDLE handle = nullptr) noexcept
        {
            if (handle_ && handle_ != INVALID_HANDLE_VALUE)
                CloseHandle(handle_);
            handle_ = handle;
        }

    private:
        HANDLE handle_;
    };
```

```
53
54      int windows_error(const char *operation)
55      {
56          std::cerr << operation << " failed, Windows error: " << GetLastError() << '\n';
57          return 1;
58      }
59
60      bool child_path(std::string &path)
61      {
62          std::array<char, 4096> buffer{};
63          const DWORD length = GetModuleFileNameA(nullptr, buffer.data(),
64                                                  static_cast<DWORD>(buffer.size()));
65          if (length == 0 || length >= buffer.size())
66              return false;
67          path.assign(buffer.data(), length);
68          const auto slash = path.find_last_of("\\/");
69          if (slash == std::string::npos)
70              return false;
71          path.resize(slash + 1);
72          path += "child.exe";
73          return true;
74      }
75  #else
76      class unique_fd
77      {
78      public:
79          explicit unique_fd(int fd = -1) noexcept : fd_(fd) {}
80          unique_fd(const unique_fd &) = delete;
81          unique_fd &operator=(const unique_fd &) = delete;
82          ~unique_fd() { reset(); }
83          int get() const noexcept { return fd_; }
84          void reset(int fd = -1) noexcept
85          {
86              if (fd_ >= 0)
87                  close(fd_);
88              fd_ = fd;
89          }
90
91      private:
92          int fd_;
93      };
94
95      bool child_path(std::string &path)
96      {
97          std::array<char, PATH_MAX> buffer{};
98          const ssize_t length = readlink("/proc/self/exe", buffer.data(), buffer.size()
                - 1);
99          if (length < 0)
100         {
101             std::perror("readlink");
102             return false;
103         }
104         buffer[static_cast<std::size_t>(length)] = '\0';
105         path.assign(buffer.data());
106         const auto slash = path.find_last_of('/');
107         if (slash == std::string::npos)
108             return false;
109         path.resize(slash + 1);
```

```
110         path += "child";
111         return true;
112     }
113 #endif
114 }
115 
116 #ifdef _WIN32
117 int main()
118 {
119     std::string filename;
120     std::cout << "Output file: ";
121     if (!std::getline(std::cin, filename) || filename.empty())
122         return 1;
123     if (filename.find('"') != std::string::npos)
124         return 1;
125 
126     std::string executable;
127     if (!child_path(executable))
128         return 1;
129 
130     SECURITY_ATTRIBUTES attributes{};
131     attributes.nLength = sizeof(attributes);
132     attributes.bInheritHandle = TRUE;
133     HANDLE input_read_raw = nullptr, input_write_raw = nullptr,
134            status_read_raw = nullptr, status_write_raw = nullptr;
135 
136     if (!CreatePipe(&input_read_raw, &input_write_raw, &attributes, 0))
137         return windows_error("CreatePipe(input)");
138     unique_handle input_read(input_read_raw), input_write(input_write_raw);
139     if (!SetHandleInformation(input_write.get(), HANDLE_FLAG_INHERIT, 0))
140         return windows_error("SetHandleInformation(input)");
141 
142     if (!CreatePipe(&status_read_raw, &status_write_raw, &attributes, 0))
143         return windows_error("CreatePipe(status)");
144     unique_handle status_read(status_read_raw), status_write(status_write_raw);
145     if (!SetHandleInformation(status_read.get(), HANDLE_FLAG_INHERIT, 0))
146         return windows_error("SetHandleInformation(status)");
147 
148     STARTUPINFOA startup{};
149     startup.cb = sizeof(startup);
150     startup.dwFlags = STARTF_USESTDHANDLES;
151     startup.hStdInput = input_read.get();
152     startup.hStdOutput = status_write.get();
153     startup.hStdError = GetStdHandle(STD_ERROR_HANDLE);
154     PROCESS_INFORMATION process{};
155     std::string command = "\"" + executable + "\" \"" + filename + "\"";
156 
157     if (!CreateProcessA(executable.c_str(), command.data(), nullptr, nullptr,
158                         TRUE, 0, nullptr, nullptr, &startup, &process))
159         return windows_error("CreateProcessA");
160     unique_handle process_handle(process.hProcess), thread_handle(process.hThread);
161     input_read.reset();
162     status_write.reset();
163 
164     std::string line;
165     for (;;)
166     {
167         std::cout << "Number: ";
```

```
168         if (!std::getline(std::cin, line))
169             break;
170         line.push_back('\n');
171         DWORD written = 0;
172         if (!WriteFile(input_write.get(), line.data(), static_cast<DWORD>(line.size()),
173                        &written, nullptr) ||
174             written != line.size())
175             break;
176         char status = 0;
177         DWORD received = 0;
178         if (!ReadFile(status_read.get(), &status, 1, &received, nullptr) || received !=
                1)
179             break;
180         print_status(status);
181         if (status == protocol::stop)
182             break;
183     }
184     input_write.reset();
185     status_read.reset();
186     WaitForSingleObject(process_handle.get(), INFINITE);
187     return 0;
188 }
189 #else
190 int main()
191 {
192     std::string filename;
193     std::cout << "Output file: ";
194     if (!std::getline(std::cin, filename) || filename.empty())
195         return 1;
196 
197     std::string executable;
198     if (!child_path(executable))
199         return 1;
200 
201     int parent_to_child[2]{};
202     int child_to_parent[2]{};
203     if (pipe(parent_to_child) < 0)
204     {
205         std::perror("pipe(parent_to_child)");
206         return 1;
207     }
208     unique_fd input_read(parent_to_child[0]), input_write(parent_to_child[1]);
209     if (pipe(child_to_parent) < 0)
210     {
211         std::perror("pipe(child_to_parent)");
212         return 1;
213     }
214     unique_fd status_read(child_to_parent[0]), status_write(child_to_parent[1]);
215 
216     const pid_t pid = fork();
217     if (pid < 0)
218     {
219         std::perror("fork");
220         return 1;
221     }
222     if (pid == 0)
223     {
224         if (dup2(input_read.get(), STDIN_FILENO) < 0 ||
```

```
225             dup2(status_write.get(), STDOUT_FILENO) < 0)
226         {
227             std::perror("dup2");
228             _exit(1);
229         }
230         input_read.reset();
231         input_write.reset();
232         status_read.reset();
233         status_write.reset();
234         execl(executable.c_str(), "child", filename.c_str(), static_cast<char *>(
                nullptr));
235         std::perror("execl");
236         _exit(1);
237     }
238 
239     input_read.reset();
240     status_write.reset();
241     std::string line;
242     for (;;)
243     {
244         std::cout << "Number: ";
245         if (!std::getline(std::cin, line))
246             break;
247         line.push_back('\n');
248         const char *data = line.data();
249         std::size_t remaining = line.size();
250         while (remaining > 0)
251         {
252             const ssize_t count = write(input_write.get(), data, remaining);
253             if (count < 0)
254             {
255                 std::perror("write");
256                 return 1;
257             }
258             data += count;
259             remaining -= static_cast<std::size_t>(count);
260         }
261         char status = 0;
262         if (read(status_read.get(), &status, 1) != 1)
263             break;
264         print_status(status);
265         if (status == protocol::stop)
266             break;
267     }
268     input_write.reset();
269     status_read.reset();
270     int status = 0;
271     waitpid(pid, &status, 0);
272     return 0;
273 }
274 #endif
```

Листинг 1: Исходный код родительского процесса

## Дочерний процесс

```cpp
#include "common.hpp"
#include "protocol.hpp"

#include <fstream>
#include <iostream>
#include <string>

int main(int argc, char* argv[]) {
    if (argc != 2) {
        std::cerr << "Usage: child <output-file>\n";
        return 1;
    }

    std::ofstream output(argv[1], std::ios::out | std::ios::trunc);
    if (!output) {
        std::cerr << "Cannot open output file\n";
        return 1;
    }

    std::string line;
    while (std::getline(std::cin, line)) {
        const auto parsed = parse_int_strict(line);
        if (!parsed) {
            std::cout << protocol::invalid << std::flush;
            continue;
        }

        const int value = *parsed;
        if (value < 0 || is_prime(value)) {
            std::cout << protocol::stop << std::flush;
            return 0;
        }
        if (value < 2) {
            std::cout << protocol::ignored << std::flush;
            continue;
        }

        output << value << '\n' << std::flush;
        if (!output) {
            std::cerr << "Cannot write to output file\n";
            return 1;
        }
        std::cout << protocol::written << std::flush;
    }
    return 0;
}
```

Листинг 2: Исходный код дочернего процесса

## Общие функции

```cpp
#include "common.hpp"

#include <cerrno>
#include <climits>
#include <cstdlib>
```

```cpp

bool is_prime(int value) {
    if (value < 2) return false;
    if (value == 2) return true;
    if (value % 2 == 0) return false;
    for (int divisor = 3; divisor <= value / divisor; divisor += 2) {
        if (value % divisor == 0) return false;
    }
    return true;
}

std::optional<int> parse_int_strict(const std::string& text) {
    char* end = nullptr;
    errno = 0;
    const long value = std::strtol(text.c_str(), &end, 10);
    if (errno != 0 || end == text.c_str() || *end != '\0' ||
        value < INT_MIN || value > INT_MAX) {
        return std::nullopt;
    }
    return static_cast<int>(value);
}
```

Листинг 3: Функции разбора числа и проверки простоты

## Заголовочные файлы

```cpp
#pragma once

#include <optional>
#include <string>

bool is_prime(int value);
std::optional<int> parse_int_strict(const std::string& text);
```

Листинг 4: Общие объявления

```cpp
#pragma once

namespace protocol {
inline constexpr char written = 'W';
inline constexpr char ignored = 'I';
inline constexpr char invalid = 'E';
inline constexpr char stop = 'S';
}
```

Листинг 5: Коды сообщений протокола обмена

## Сборка

```
cmake_minimum_required(VERSION 3.16)
project(os_lab2_variant5 LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
```

```
add_library(common STATIC inc/common.cpp)
target_include_directories(common PUBLIC inc/include)
add_executable(parent inc/parent.cpp)
target_link_libraries(parent PRIVATE common)
add_executable(child inc/child.cpp)
target_link_libraries(child PRIVATE common)
set_target_properties(parent child PROPERTIES RUNTIME_OUTPUT_DIRECTORY ${
    CMAKE_BINARY_DIR}/bin)
if(MSVC)
  target_compile_options(common PRIVATE /W4 /permissive-)
  target_compile_options(parent PRIVATE /W4 /permissive-)
  target_compile_options(child PRIVATE /W4 /permissive-)
else()
  target_compile_options(common PRIVATE -Wall -Wextra -Wpedantic -Werror)
  target_compile_options(parent PRIVATE -Wall -Wextra -Wpedantic -Werror)
  target_compile_options(child PRIVATE -Wall -Wextra -Wpedantic -Werror)
endif()
```

Листинг 6: Файл конфигурации CMake

## Проверка системных вызовов

Для просмотра системных вызовов Linux-версии применяется утилита `strace`. Команда запуска:

```
printf 'result.txt\n4\n9\n7\n' | strace -f \
-e trace=process,pipe,dup2,read,write,close \
./build/bin/parent
```

В выводе должны присутствовать системные вызовы `pipe()`, `fork()`, `dup2()`, `execve()`, `read()`, `write()`, `close()` и `wait4()`. Вызов `execve()` соответствует запуску дочерней программы, а `wait4()` является системной реализацией библиотечной функции `waitpid()`.